Atlas-Embedding Proofs (AEPs)

Runnable hologram clients of Atlas facts

Purpose (one-liner)

Embeddings are coordinates; the Atlas ISA gives executable semantics. **AEPs** turn Atlas facts into small, portable programs ("hologram clients") that *prove* those facts by running invariant-checked kernels on any backend.

> **TL;DR to say out loud:** "Embeddings tell you where things live; the ISA tells you how they're allowed to interact — and lets any domain head run those interactions on any hardware."

What an AEP is

An **Atlas-Embedding Proof (AEP)** is a tiny, portable program that: 1. **States a fact** about Atlas in terms of ISA invariants/structures (e.g., *mirror-safe*, *unity-neutral*, *boundary window*, *phase window*, *skeleton respect*). 2. **Implements the fact** as an Atlas kernel **plus metadata** (e.g., `mirror_safe`, `unity_neutral`, `uses_boundary`, `phase_begin/phase_span`, `classes_mask`). 3. **Produces verifiable evidence** via standardized verification hooks (e.g., resonance snapshots, Δ-channel, boundary/phase probes). Launch must **fail deterministically** if the invariant is violated. 4. **Runs anywhere** thanks to the ISA's target-agnostic model and normative legalization to CPU/GPU/TPU/DSP/FPGA.

Minimal AEP template

Files: - `aep.toml` — declarative claim & parameters - `kernel.atlas` — the proof kernel (plus attributes) - `runner.py` — portable harness that launches the kernel and checks evidence

`aep.toml` (example)

```
name = "aep_unity_vecadd"
claim = "unity_neutral"          # one of: mirror_safe, unity_neutral,
boundary_window, phase_window, skeleton_respect
witness = "resonance_snapshot"   # or: delta_channel, boundary_probe,
phase_probe

# Optional parameters depending on claim
classes_mask = ["C96:scalar"]
boundary_window = { x=[16,47], y=[8,31] }
```

```
phase_begin = 128
phase_span = 32
```

`kernel.atlas` **(pseudocode)**

```
// Attributes declare the invariants we claim
.attributes [unity_neutral, mirror_safe]

// ABI: read PB (plain-old-data) for inputs
pb.load A, B, OUT, N

// Canonical grid/block/lanes; ISA guarantees portable scheduling & barriers
for gid in grid(N):
  OUT[gid] = A[gid] + B[gid]
  // No RES writes in a unity-neutral claim
end
```

`runner.py` **(sketch)**

```python
import atlas
from pathlib import Path

mod = atlas.load_module("kernel.atlas")
rt = atlas.Runtime()  # selects a target or uses default; e.g., CPU/GPU/TPU

# Prepare PB
A, B = rt.buffer.randn(1_000_000), rt.buffer.randn(1_000_000)
OUT = rt.buffer.zeros_like(A)

# Take resonance snapshot (pre)
pre = rt.snapshot_resonance()

# Launch
rt.launch(mod, grid=(A.size,), pb=dict(A=A, B=B, OUT=OUT, N=A.size))

# Take resonance snapshot (post)
post = rt.snapshot_resonance()

delta = post - pre
assert delta.is_zero(), f"Unity-neutrality violated: ΔR={delta}"
print("PASS: unity-neutral vec_add")
```

Three example AEPs

1) Unity-neutrality of `vec_add` (smoke test)

- **Claim:** Kernel is `unity_neutral` (no net contribution to resonance accumulator).
- **Kernel:** Canonical `vec_add` marked `[unity_neutral]`. No `RES.*` writes.
- **Witness:** Compare resonance snapshots pre/post (or read Δ-channel). **Pass** iff ΔR = 0; else **fault**.

2) Mirror-safety under class conjugation

- **Claim:** For a `[mirror_safe]` kernel, substituting `c → mirror(c)` leaves observable effects invariant.
- **Kernel:** Read class labels with `CLS.GET`; run compute on `c` and on `mirror(c)`; compare outputs.
- **Witness:** Byte-equality of outputs; a mismatch is a **counter-example**.

3) Boundary-lens footprint compliance

- **Claim:** All neighbor/projection accesses respect a declared `boundary_window` on a toroidal surface.
- **Kernel:** Mark `[uses_boundary]`; perform mapped accesses with `BOUND.MAP` and probe edges.
- **Witness:** Runtime **fault** on any out-of-range access; otherwise emit a coverage map artifact for audit.

Attaching “arbitrary domain heads”

A **domain head** is a lightweight adapter that maps domain objects to Atlas PB/tensors **without** changing invariant semantics.

- **Core (proof):** Pure ISA — kernels, attributes, and verification hooks.
- **Domain head:** Interprets buffers (NLP embeddings, graphs, PDE fields, imaging) and packs/unpacks PB; it never alters invariants.
- **Result:** Same AEP runs across CPU/GPU/TPU/DSP/FPGA; the ISA preserves ordering, barriers, memory spaces, and checks.

Suggested repo layout

```
atlas-embedding-proofs/
  aep_unity_vecadd/
    aep.toml
    kernel.atlas
    runner.py
    README.md
```

```
  aep_mirror_safety_conjugation/
    aep.toml
    kernel.atlas
    runner.py
    README.md
  aep_boundary_window_probe/
    aep.toml
    kernel.atlas
    runner.py
    README.md
  common/
    domain_heads/
      nlp_head.py
      graph_head.py
      pde_head.py
    runtime/
      resonance_snapshot.py
      delta_channel.py
```

Quickstart (for contributors)

1. **Copy** `aep_template/` → `aep_<your_claim>/`.
2. Edit `aep.toml` to set `claim`, `witness`, and any parameters.
3. Implement `kernel.atlas` and mark attributes matching the claim.
4. Write `runner.py` to capture evidence (snapshot, Δ-channel, probes) and assert the claim.
5. Run on at least **two backends** (e.g., CPU + GPU) to demonstrate portability.
6. Commit artifacts: pass/fail logs, resonance deltas, coverage maps.
7. Open a PR with a short README stating the fact and how the proof demonstrates it.

Why this convinces a crowd

- **Executable semantics:** Facts aren't essays; they're launches that must pass or fault.
- **Hardware-agnostic:** Same proof, many targets — thanks to the ISA's generalized compute & checks.
- **Uniform verification:** Resonance snapshots, Δ-channel, and boundary/phase probes standardize evidence.

Handy mental picture

```
[Your Domain Head]
   ↓ emits: kernels + invariants (classes_mask, boundary, phase, unity)
[ Atlas ISA (compute + checks) ]
```

```
   ↓ legalization preserves ordering & invariants
[ CPU ] [ GPU ] [ TPU/PJRT ] [ DSP/SIMD ] [ FPGA ]
```

Notes

- Keep AEPs minimal and single-purpose; prefer several tiny proofs over one omnibus proof.
- Favor deterministic runners; when nondeterminism is inherent, bound it (fixed seeds, bounded phase windows) and document it.
- Treat any **invariant violation** from the runtime as a **valid counter-example** to the claim.